

Prompt:

8/17/2026, 10:07:53 PM

lanjut Tahap U

Response:

8/17/2026, 10:07:54 PM

Remembering

Reviewed prior Tahap structure and preserved mathematical trading conventions

Worked for 7s

Tahap U - Execution & Order-Routing Engine

Kita sekarang masuk ke lapisan yang membuat sistem kita berpindah dari **quantitative strategy** menjadi sesuatu yang lebih mendekati **trading infrastructure**.

Pada Tahap T kita sudah menentukan:

berapa besar posisi, SL , TP , $riskbudget$, dan kapan posisi harus dipertahankan/diubah/ditutup.

Tetapi kita masih mempunyai satu masalah:

Bagaimana order tersebut benar-benar masuk ke pasar tanpa membuat asumsi backtest yang terlalu ideal?

Di dunia nyata, ketika model mengatakan:

BUY q units at P

belum tentu kita mendapatkan:

q units at P .

Kita bisa mendapatkan:

$q_1 < q$

pada harga berbeda, sebagian order tidak terisi, harga bergerak sebelum order sampai ke exchange, spread berubah, atau order kita terkena adverse selection.

Maka Tahap U akan membangun:

Execution Engine

yang berada di antara:

Decision Engine → Execution Engine → Market

U.1. Prinsip utama

Kita harus memisahkan dua objective:

Portfolio/strategy objective

$$\max E[U]$$

dengan tujuan mendapatkan alpha.

Execution objective

$$\min E[C_{\text{execution}}]$$

dengan tujuan mendapatkan exposure yang diinginkan **semurah dan seaman mungkin**.

Jadi:

Strategy mencari opportunity

sedangkan:

Execution mencari cara terbaik untuk merealisasikan opportunity tersebut.

U.2. Mengapa harga model bukan harga eksekusi?

Misalkan model menghasilkan:

$$P_t^{model} = 100.$$

Kita ingin:

$$q = 10.$$

Order book:

Ask	Volume
100.01	2
100.02	3
100.05	5
100.10	10

Jika kita langsung market buy 10 unit, maka kita memperoleh:

$$2(100.01) + 3(100.02) + 5(100.05).$$

Average execution:

$$P_{avg} = \frac{2(100.01) + 3(100.02) + 5(100.05)}{10}.$$

Hasilnya:

$$P_{avg} = 100.033.$$

Jadi:

$$P_{avg} - P_{model} = 0.033.$$

Itulah salah satu bentuk **execution slippage**.

U.3. Decomposition transaction cost

Saya ingin kita mendefinisikan:

$$C_{execution} = C_{spread} + C_{fee} + C_{slippage} + C_{impact} + C_{latency} + C_{opportunity}$$

Masing-masing mempunyai makna berbeda.

U.4. Spread cost

Mid-price:

$$M_t = \frac{A_t + B_t}{2}.$$

Jika kita membeli menggunakan ask:

$$C_{spread} = A_t - M_t = \frac{A_t - B_t}{2}.$$

Jika menjual menggunakan bid:

$$C_{spread} = M_t - B_t.$$

Jadi secara sederhana:

$$C_{spread} = \frac{Spread}{2}.$$

Untuk round trip:

$$C_{spread,RT} \approx Spread.$$

U.5. Fee

Misalkan fee:

$$f.$$

Untuk notional:

$$N = qP,$$

maka:

$$C_{fee} = fN.$$

Untuk round trip:

$$C_{fee,RT} = f_e N_e + f_x N_x.$$

Kita tidak boleh hanya memasukkan fee entry.

U.6. Slippage

Definisikan implementation shortfall terhadap decision price:

$$P_0 = P_{decision}.$$

Untuk long:

$$IS = P_{execution} - P_0.$$

Untuk short:

$$IS = P_0 - P_{execution}.$$

Secara umum:

$$IS_t = s_t(P_{exec} - P_{decision})$$

dengan:

$$s_t = \begin{cases} +1, & \text{long} \\ -1, & \text{short.} \end{cases}$$

U.7. Implementation Shortfall

Untuk seluruh order:

$$IS = \frac{\sum_i q_i P_i}{Q} - P_0$$

untuk long.

Namun implementation shortfall yang lebih lengkap adalah:

$$IS = \underbrace{P_{arrival} - P_{decision}}_{\text{delay}} + \underbrace{P_{execution} - P_{arrival}}_{\text{execution}} + \underbrace{fees}_{\text{cost}}$$

Ini sangat berguna untuk mengevaluasi execution engine kita.

U.8. Arrival price

Kita membutuhkan sebuah referensi:

$$P_{arrival}.$$

Biasanya:

$$P_{arrival} = M_{t_0}.$$

Kemudian kita bertanya:

Berapa biaya yang sebenarnya kita keluarkan sejak keputusan dibuat?

Bukan sekadar:

Apakah order akhirnya profit?

U.9. Latency

Misalkan:

t_d = decision time,

t_s = signal sent,

t_e = exchange received.

Maka:

$$L = t_e - t_d.$$

Dalam market yang cepat, selama:

$$L > 0,$$

harga dapat berubah.

Maka:

$$P_{\text{market}}(t_e) \approx P_{\text{market}}(t_d).$$

U.10. Expected latency cost

Misalkan instantaneous price drift:

$$\mu_t.$$

Maka approximation:

$$E[\Delta P | L] \approx \mu_t L.$$

Tetapi untuk short horizon, stochastic component lebih penting:

$$\Delta P = \mu L + \sigma \sqrt{L} \epsilon.$$

Sehingga:

$$\boxed{\text{Var}(\Delta P | L) = \sigma^2 L.}$$

Semakin besar latency:

$$L \uparrow \Rightarrow \text{execution uncertainty} \uparrow .$$

U.11. Market impact

Ini bagian yang jauh lebih menarik.

Jika kita membeli:

$$q$$

unit, kita menghabiskan liquidity pada ask.

Harga rata-rata dapat dimodelkan:

$$P_{exec} = P_{mid} + SpreadCost + Impact(q).$$

Model sederhana:

$$Impact(q) = Y \sigma \left(\frac{q}{V} \right)^\alpha$$

dengan:

- Y : impact coefficient,
- σ : volatility,
- V : volume,
- q : order size,
- α : impact exponent.

U.12. Mengapa volatility masuk?

Karena market yang volatile cenderung mempunyai:

- wider effective spread,
- lower certainty of liquidity,
- larger price response terhadap order.

Maka:

$$Impact \propto \sigma.$$

Ini berarti sistem kita secara otomatis menganggap execution menjadi lebih mahal ketika market menjadi lebih volatile.

U.13. Liquidity ratio

Kita dapat mendefinisikan:

$$ParticipationRate = \frac{q}{V_{window}}.$$

Misalnya:

$$q = 1000$$

dan:

$$V_{window} = 1,000,000.$$

Maka:

$$ParticipationRate = 0.1\%.$$

Semakin besar participation rate:

$$Impact \uparrow .$$

U.14. Order Book Depth

Untuk short-term trading, historical volume saja tidak cukup.

Kita ingin mengetahui:

$$D_k^a = \sum_{i=1}^k v_i^a$$

untuk k level ask.

Begitu pula:

$$D_k^b = \sum_{i=1}^k v_i^b.$$

U.15. Market order simulation

Misalkan:

$$Q = 100.$$

Order book ask:

<i>Price</i>	<i>Volume</i>
100.01	20
100.02	30
100.03	50

Maka:

$$Q_1 = 20,$$

$$Q_2 = 30,$$

$$Q_3 = 50.$$

Execution price:

$$P_{VWAP} = \frac{20(100.01) + 30(100.02) + 50(100.03)}{100}.$$

Inilah yang seharusnya dilakukan simulator kita.

Bukan:

$$P_{execution} = 100.01.$$

U.16. VWAP

Secara umum:

$$VWAP = \frac{\sum_i q_i P_i}{\sum_i q_i}.$$

Untuk market order:

$$P_{exec} = VWAP_{consumed\ book}.$$

U.17. Limit order

Sekarang lebih menarik.

Misalkan:

$$P_{limit} = 99.95.$$

Order tidak langsung dieksekusi.

Kita memiliki:

$$P(fill)$$

sebagai random variable.

Jadi:

$$F \sim \text{Bernoulli}(p_{fill}).$$

Jika:

$$F = 1,$$

order filled.

Jika:

$$F = 0,$$

order tidak mendapat exposure.

U.18. Fill probability

Kita dapat membangun:

$$p_{fill} = f(d, Q_{ahead}, I, \sigma, V, T)$$

dengan:

$$d = |P_{limit} - P_{mid}|$$

dan:

Q_{ahead} = queue volume di depan order kita.

U.19. Queue position

Misalkan:

$$Q_{ahead} = 500.$$

Order kita:

$$Q_{ours} = 100.$$

Jika volume yang dieksekusi pada harga tersebut baru:

$$300,$$

maka kita belum filled.

Jika:

$$V_{executed} = 550,$$

maka secara ideal:

$$500$$

untuk queue di depan dan:

$$50$$

untuk order kita.

Sehingga:

$$Q_{filled} = 50.$$

U.20. Partial fill

Maka:

$$0 \leq q_{filled} \leq q_{target}.$$

Kita definisikan:

$$FillRatio = \frac{q_{filled}}{q_{target}}.$$

Misalnya:

$$FillRatio = 0.35.$$

Artinya hanya 35% posisi yang berhasil diperoleh.

U.21. Partial fill adalah decision problem

Kita kemudian harus memutuskan:

$$a \in \{WAIT, CANCEL, REPRICE, MARKET_FILL\}.$$

Misalnya:

$$q_{remaining} = 650.$$

Kita harus menghitung expected value dari masing-masing action.

U.22. Optimal cancellation

Misalkan:

$$EV_{wait} = p_{fill}EV_{trade} - C_{waiting}.$$

Sedangkan:

$$EV_{market} = EV_{trade} - C_{market}.$$

Kita memilih:

$$a^* = \arg \max_a EV(a).$$

Ini berarti cancellation bukan aturan arbitrer seperti:

"Cancel setelah 5 menit."

Tetapi:

$cancel \text{ ketika } expected \text{ value waiting} < alternative.$

U.23. Adverse selection

Ini salah satu konsep paling penting untuk limit order.

Misalkan kita memasang buy limit:

$$P_{limit} = 100.$$

Order kita filled.

Mengapa?

Bisa jadi karena:

$$P_{future} < 100.$$

Dengan kata lain:

Kita mendapatkan fill justru ketika informasi pasar sedang buruk bagi kita.

Jadi:

$$P(fill)$$

dan:

$$R_{future}$$

tidak independen.

U.24. Conditional fill model

Kita harus memodelkan:

$E[R_{future} \mid fill, S_t]$

bukan:

$$E[R_{future}|S_t].$$

Ini perbedaan yang sangat penting.

U.25. Expected value limit order

Maka:

$$EV_{limit} = P(\text{fill})E[R|\text{fill}] + P(\text{no fill})E[R|\text{no fill}] - \text{Cost}.$$

Jika no-fill berarti kita kehilangan opportunity:

$$E[R|\text{no fill}]$$

juga harus diperhitungkan.

U.26. Market vs limit order

Sekarang kita bisa membuat decision:

$$EV_{market} \quad \text{vs} \quad EV_{limit}.$$

Jika:

$$EV_{market} > EV_{limit},$$

gunakan marketable order.

Jika:

$$EV_{limit} > EV_{market},$$

gunakan limit.

Ini lebih sophisticated daripada:

"Selalu gunakan limit order agar fee murah."

U.27. Order type sebagai optimization variable

Kita definisikan:

$$o_t \in \{MARKET, LIMIT, POST_ONLY, IOC, FOK, TWAP, VWAP\}.$$

Kemudian:

$$o_t^* = \arg \max_o EV(o|S_t).$$

U.28. TWAP

Untuk order besar:

$$Q.$$

Daripada membeli semuanya:

$$Q$$

sekaligus, kita bagi:

$$Q_1, Q_2, \dots, Q_N.$$

dengan:

$$\sum_i Q_i = Q.$$

TWAP mengatur execution relatif merata terhadap waktu.

U.29. VWAP execution

Jika expected market volume:

$$V_1, V_2, \dots, V_N,$$

maka:

$$Q_i = Q \frac{V_i}{\sum_j V_j}.$$

Sehingga:

$$\boxed{\frac{Q_i}{Q} = \frac{V_i}{V_{total}}}.$$

Kita mengikuti volume market.

U.30. Tetapi kita bisa membuat execution adaptif

Misalkan alpha kita:

$$\alpha_t.$$

Jika:

$$\alpha_t \gg 0,$$

kita ingin memperoleh exposure lebih cepat.

Jika:

$$\alpha_t \approx 0,$$

kita dapat lebih sabar.

Jadi execution schedule:

$$Q_t = f(\alpha_t, \sigma_t, L_t, liquidity_t).$$

U.31. Urgency

Definisikan:

$$u_t \in [0, 1].$$

Dengan:

$$u_t = 0$$

berarti:

patient.

Sedangkan:

$$u_t = 1$$

berarti:

urgent.

Kemudian:

$$u_t = f(\text{alpha}, \text{time decay}, \text{volatility}, \text{fill probability}).$$

U.32. Time decay alpha

Misalkan expected alpha:

$$\alpha(\tau) = \alpha_0 e^{-\kappa\tau}.$$

Semakin lama kita menunggu:

$$\tau \uparrow$$

maka:

$$\alpha(\tau) \downarrow .$$

Maka urgency meningkat:

$$u(\tau) \uparrow .$$

U.33. Ini menghasilkan execution policy

$$Q_t^* = \pi_{\text{execution}}(\alpha_t, \sigma_t, \text{liquidity}_t, \text{spread}_t, \text{queue}_t, \text{latency}_t, \text{time}_t).$$

Jadi execution sendiri menjadi sebuah **policy optimization problem**.

U.34. Market impact vs opportunity cost

Ini inti dari optimal execution.

Kalau kita eksekusi terlalu cepat:

$$\text{Impact} \uparrow .$$

Kalau terlalu lambat:

$$\text{OpportunityCost} \uparrow .$$

Maka kita mempunyai:

$$\text{TotalCost} = \text{ImpactCost} + \text{OpportunityCost}.$$

Dan:

$$\tau^* = \arg \min_{\tau} [\text{Impact}(\tau) + \text{OpportunityCost}(\tau)].$$

U.35. Model sederhana

Misalkan:

$$\text{ImpactCost}(\tau) = \frac{a}{\tau^\gamma}$$

dan:

$$\text{OpportunityCost}(\tau) = b\tau.$$

Maka:

$$C(\tau) = \frac{a}{\tau^\gamma} + b\tau.$$

Turunan:

$$\frac{dC}{d\tau} = -a\gamma\tau^{-(\gamma+1)} + b.$$

Set:

$$\frac{dC}{d\tau} = 0.$$

Maka:

$$a\gamma\tau^{-(\gamma+1)} = b.$$

Sehingga:

$$\tau^* = \left(\frac{a\gamma}{b}\right)^{1/(\gamma+1)}.$$

Ini merupakan contoh sederhana bagaimana **execution horizon dapat ditentukan secara matematis**.

U.36. Hubungan dengan alpha

Jika:

$$b \propto |\alpha|,$$

maka:

$$|\alpha| \uparrow \Rightarrow \tau^* \downarrow.$$

Artinya:

Alpha kuat → eksekusi lebih cepat.

Sebaliknya:

$$|\alpha| \downarrow \Rightarrow \tau^* \uparrow.$$

Ini sangat masuk akal secara finansial.

U.37. Sekarang kita masukkan risk

Execution tidak boleh meningkatkan risk melebihi budget.

Misalkan target:

$$R_{target}.$$

Setiap delay memberikan exposure uncertainty:

$$Var(R_{delay}) = \sigma^2 \tau.$$

Maka:

$$\sigma^2 \tau \leq R_{uncertainty,max}^2.$$

Ini menghasilkan upper bound terhadap execution horizon.

U.38. Execution constraint

Jadi:

$$\tau \leq \frac{R_{uncertainty,max}^2}{\sigma^2}.$$

Market semakin volatile:

$$\sigma \uparrow$$

maka:

$$\tau_{max} \downarrow .$$

Sekali lagi:

Volatility tinggi → kita tidak boleh terlalu santai saat eksekusi.

U.39. Exchange/venue selection

Untuk sistem multi-market, kita mungkin mempunyai:

$$V = \{v_1, v_2, \dots, v_n\}.$$

Setiap venue memiliki:

$$C_v = fee_v + spread_v + impact_v + latency_v + risk_v.$$

Kita memilih:

$$v^* = \arg \min_v E[C_v].$$

U.40. Tetapi venue juga memiliki liquidity

Maka:

$$C_v = C_v(q).$$

Sehingga:

$$v^* = \arg \min_v C_v(q).$$

Dan untuk order besar, kita bahkan dapat melakukan split:

$$q_1 + q_2 + \dots + q_n = q.$$

U.41. Venue allocation sebagai optimization problem

Misalkan:

$$x_v$$

adalah fraction order ke venue v .

Maka:

$$\sum_v x_v = 1,$$

$$x_v \geq 0.$$

Objective:

$$\min_{\mathbf{x}} \sum_v C_v(x_v q)$$

subject to:

$$\sum_v x_v = 1.$$

Ini sudah merupakan **capital allocation problem**, hanya saja pada level execution.

U.42. Dan di sinilah QUBO dapat muncul lagi

Jika keputusan venue dibatasi menjadi:

$$x_v \in \{0, 1\},$$

kita dapat membangun:

$$QUBO_{venue}.$$

Misalnya:

$$\min \sum_v c_v x_v + \lambda \left(\sum_v x_v - 1 \right)^2.$$

Penalty memastikan hanya satu venue dipilih.

U.43. Jika multi-venue splitting

Kita bisa menggunakan discrete quantity:

$$x_{v,k} \in \{0, 1\}$$



untuk memilih:

venue v mendapatkan tranche k .

Kemudian objective:

$$\min [ExecutionCost + Impact + LatencyRisk + Penalty].$$

Ini menjadi candidate problem untuk QUBO.

U.44. Tetapi kita harus disiplin

Jangan menggunakan QAOA hanya karena kita bisa.

Jika:

QUBO

dapat diselesaikan dengan:

- MILP,
- dynamic programming,
- quadratic programming,
- greedy,
- simulated annealing,

maka semuanya harus menjadi benchmark.

Prinsip proyek kita tetap:

Quantum algorithm $\nexists$ automatic advantage.

U.45. Event-driven execution simulator

Tahap U harus diwujudkan dalam simulator event-driven.

Event:

$E_t \in \{MARKET_UPDATE, ORDER_SUBMIT, ORDER_ACK, PARTIAL_FILL, FULL_FILL, CANCEL, RE$



U.46. State machine

Order memiliki state:

$NEW \rightarrow SUBMITTED \rightarrow ACKNOWLEDGED \rightarrow PARTIALLY_FILLED \rightarrow FILLED$

atau:

$$SUBMITTED \rightarrow CANCELLED$$

atau:

$$SUBMITTED \rightarrow REJECTED.$$

Ini penting agar backtest tidak melakukan:

```
signal = buy
price = close
filled = true
```

secara instan.

U.47. Latency-aware simulator

Kita harus mempunyai:

 $t_{decision}$ t_{submit} $t_{exchange}$ $t_{fill}.$

Dengan:

$$t_{decision} < t_{submit} < t_{exchange} \leq t_{fill}.$$

Kemudian harga yang digunakan untuk fill adalah:

$$P(t_{fill}),$$

bukan:

$$P(t_{decision}).$$

U.48. Ini mencegah look-ahead

Misalnya candle 10:00-10:01.

Jika signal baru diketahui pada:

10 : 01 : 00,

maka simulator tidak boleh mengisi order menggunakan:

10 : 00 : 30.

Ini merupakan:

point-in-time execution.

U.49. Untuk sistem kita, ini wajib

Karena kita menggunakan:

- crypto,
- saham internasional,
- gold,
- USD/DXY,
- market factors,

timestamp antar instrumen bisa berbeda.

Jadi kita membutuhkan:

Event Time

bukan sekadar row number.

U.50. Multi-asset synchronization

Misalnya:

$$BTC_t$$

tersedia setiap detik.

Tetapi:

$$DXY_t$$

hanya tersedia ketika market FX buka.

Dan:

$$SP X_t$$

memiliki jam perdagangan tertentu.

Kita tidak boleh mengisi data masa depan secara sembarangan.

Untuk setiap asset i :

$$t_i^{last} = \max\{\tau \leq t : Data_i(\tau) \text{ available}\}.$$

Kemudian:

$$X_i(t) = Data_i(t_i^{last}).$$

Tetapi kita harus menyimpan timestamp aslinya:

$$age_i(t) = t - t_i^{last}.$$

U.51. Staleness penalty

Jika:

$$age_i(t)$$

terlalu besar:

$$age_i(t) > T_{max},$$

maka:

$$Data_i(t) = INVALID.$$

Ini penting untuk factor seperti:

$$DXY, Gold, VIX.$$

U.52. Cross-asset latency

Misalkan:

$$BTC$$

bergerak sekarang.

Model menggunakan:

$$DXY$$

yang terakhir diperbarui 20 menit lalu.

Kita tidak boleh berpura-pura bahwa DXY merupakan informasi real-time.

Maka state kita harus menyimpan:

$$(age_{BTC}, age_{DXY}, age_{Gold}, age_{SPX}, \dots)$$

sebagai bagian dari market state.

U.53. Execution state

Saya menyarankan state lengkap menjadi:

$$S_t = [MarketState, PortfolioState, OrderState, ExecutionState].$$

Dengan:

$$ExecutionState = [spread, depth, queue, latency, fillRate, impact, venue].$$

U.54. Execution alpha

Ini konsep yang sangat menarik.

Kita memiliki:

$$\alpha_{strategy}$$

dan:

$$\alpha_{execution}$$

Strategy alpha mengatakan:

"Saya memperkirakan harga naik."

Execution alpha mengatakan:

"Saya memperkirakan sekarang adalah waktu/venue/order type terbaik untuk mendapatkan exposure tersebut."

Jadi:

$$TotalValue = StrategyAlpha - ExecutionCost.$$

U.55. Contoh

Strategy:

$$E[R] = 20bps.$$

Execution A:

$$Cost = 5bps.$$

Maka:

$$Net = 15bps.$$

Execution B:

$$Cost = 18bps.$$

Maka:

$$Net = 2bps.$$

Keduanya menggunakan signal yang sama.

Tetapi execution engine menentukan apakah trade tersebut masih layak.

U.56. Trade admission

Maka sebelum order:

$$E[R_{net}] = E[R_{gross}] - E[C_{execution}]$$

dan trade hanya diterima jika:

$$E[R_{net}] > H.$$

Dengan H sebagai hurdle rate.

U.57. Ini sangat penting untuk short-term trading

Semakin pendek holding period:

$$T_{hold} \downarrow,$$

semakin besar kemungkinan:

$$C_{execution}$$

mengonsumsi sebagian besar:

$$E[R_{gross}].$$

Maka:

Short horizon $\Rightarrow$ execution becomes part of alpha.

U.58. Backtest metric baru

Kita tidak cukup menggunakan:

- Sharpe ratio,
- CAGR,
- max drawdown.

Kita harus mengukur execution:

Implementation Shortfall

$IS.$

Fill Ratio

$$FR = \frac{Q_{filled}}{Q_{target}}.$$

Slippage

$$S = P_{exec} - P_{arrival}.$$

Market Impact

$$MI = P_{exec} - P_{mid}.$$

Latency

$$L = t_{fill} - t_{decision}.$$

Cancellation Rate

$$CR = \frac{N_{cancel}}{N_{submitted}}.$$

U.59. Adverse Selection Metric

Untuk limit order:

$$AS(\Delta) = E[P_{future} - P_{fill} | fill]$$

untuk long.

Jika:

$$AS < 0,$$

maka fill cenderung diikuti harga turun.

Itu berarti:

limit order strategy mengalami adverse selection.

U.60. Implementation Shortfall decomposition

Kita dapat memecah:

$IS = Delay + Spread + Impact + Timing + Fees.$

Dengan ini kita dapat mengetahui **mengapa strategy kehilangan performance**.

Misalnya:

$$Gross\ Alpha = 30bps.$$

Tetapi:

$$Delay = 2bps$$

$$Spread = 4bps$$

$$Impact = 6bps$$

$$Fees = 3bps.$$

Maka:

$$Net = 15bps.$$

U.61. Dan ini membuka optimasi yang sangat menarik

Jika kita melihat:

$$Impact = 6bps,$$

kita bisa mengoptimalkan:

$$execution\ schedule.$$

Jika:

$$Delay = 2bps,$$

kita optimalkan:

$$latency.$$

Jika:

$$Spread = 4bps,$$

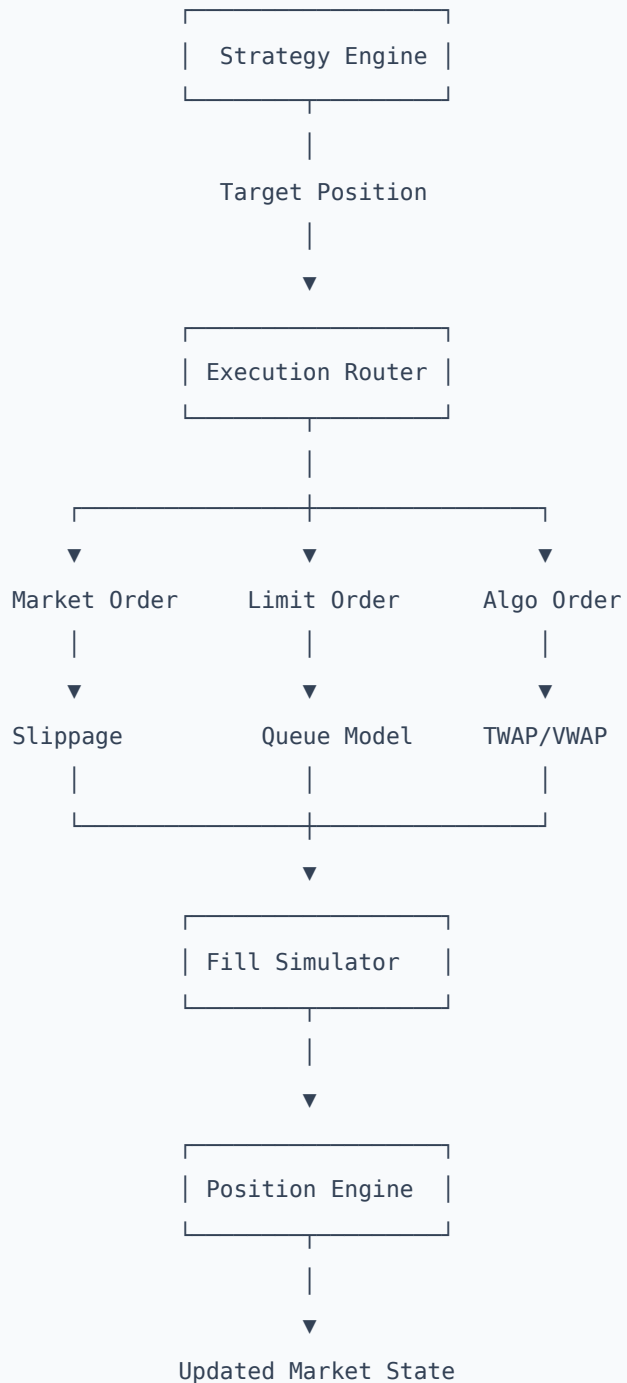
kita optimalkan:

$$order\ type.$$

Jadi diagnostic decomposition langsung memberikan roadmap improvement.

U.62. Arsitektur Tahap U

Saya sarankan:



U.63. Dan kita sekarang mempunyai dua optimizer

Ini menurut saya salah satu hasil terpenting dari Tahap U.

Optimizer I - Investment/Trading Optimizer

Menentukan:

What should we trade?

Contoh:

QUBO, Markowitz, Ising, QAOA.

Optimizer II - Execution Optimizer

Menentukan:

How should we trade it?

Contoh:

market/limit, venue, timing, ordersplitting, TWAP/VWAP.

U.64. Unified objective

Akhirnya kita bisa menuliskan:

$$\max_{\pi, \epsilon} E[U(W_T)]$$

dengan:

π = strategy policy

dan:

ϵ = execution policy.

Subject to:

$$\begin{aligned} Risk &\leq R_{max}, \\ Leverage &\leq L_{max}, \\ Liquidity &\geq L_{min}, \\ Impact &\leq I_{max}, \\ Drawdown &\leq D_{max}, \\ ExecutionCost &\leq C_{max}. \end{aligned}$$

Jadi bukan lagi:

maximize trading signal.



Tetapi:

maximize realized risk-adjusted utility after execution.

U.65. Hubungan dengan sistem keseluruhan

Sekarang rantai kita menjadi:

Market → *State*
→ *Factor* → *Alpha*
→ *Regime* → *Risk*
→ *Portfolio*
→ *QUBO/Optimization*
→ *TargetPosition*
→ *ExecutionOptimization*
→ *Order*
→ *Fill*
→ *Position*
→ *NewState.*

Dan karena:

New State

kembali masuk ke model, maka sistem benar-benar menjadi:

closed-loop adaptive quantitative trading system.

U.66. Hal yang menurut saya paling penting dari Tahap U

Ada satu perubahan paradigma yang ingin saya tetapkan untuk implementasi kita nanti:

Backtest tidak boleh lagi menganggap bahwa "signal menghasilkan trade".

Yang benar adalah:

Signal → *Target Position* → *Execution Decision* → *Order* → *Fill* → *Actual Position.*

Dengan demikian kita dapat membedakan tiga hal:

Paper Alpha

vs.

Executable Alpha

vs.

Realized Alpha.

Ketiganya bisa berbeda secara signifikan.

U.67. Dan ini menjadi fondasi Tahap V

Setelah Tahap U, kita sudah memiliki hampir semua mekanisme yang dibutuhkan untuk membuat **simulator trading event-driven yang realistis**.

Maka Tahap berikutnya seharusnya bukan sekadar menambahkan indikator atau model ML baru. Yang jauh lebih penting adalah:

Tahap V - Event-Driven Backtesting & Market Replay Engine

Di sana kita akan menyatukan:

Historical Data + Market State + Alpha + Portfolio Optimizer + Risk Engine + Execution Engine + Order Book

menjadi **satu simulator deterministik** yang dapat menjawab pertanyaan paling penting:

"Jika seluruh sistem kita benar-benar dijalankan pada informasi yang tersedia pada waktu t , dengan order yang benar-benar harus melewati execution process, berapa performance yang sebenarnya akan diperoleh?"

Dan itu juga akan menjadi tempat kita memastikan tidak terjadi **look-ahead bias, survivorship bias, data leakage, unrealistic fills, atau hindsight optimization**-sebelum hasil sistem kita mulai dibandingkan dengan benchmark klasik maupun QAOA.